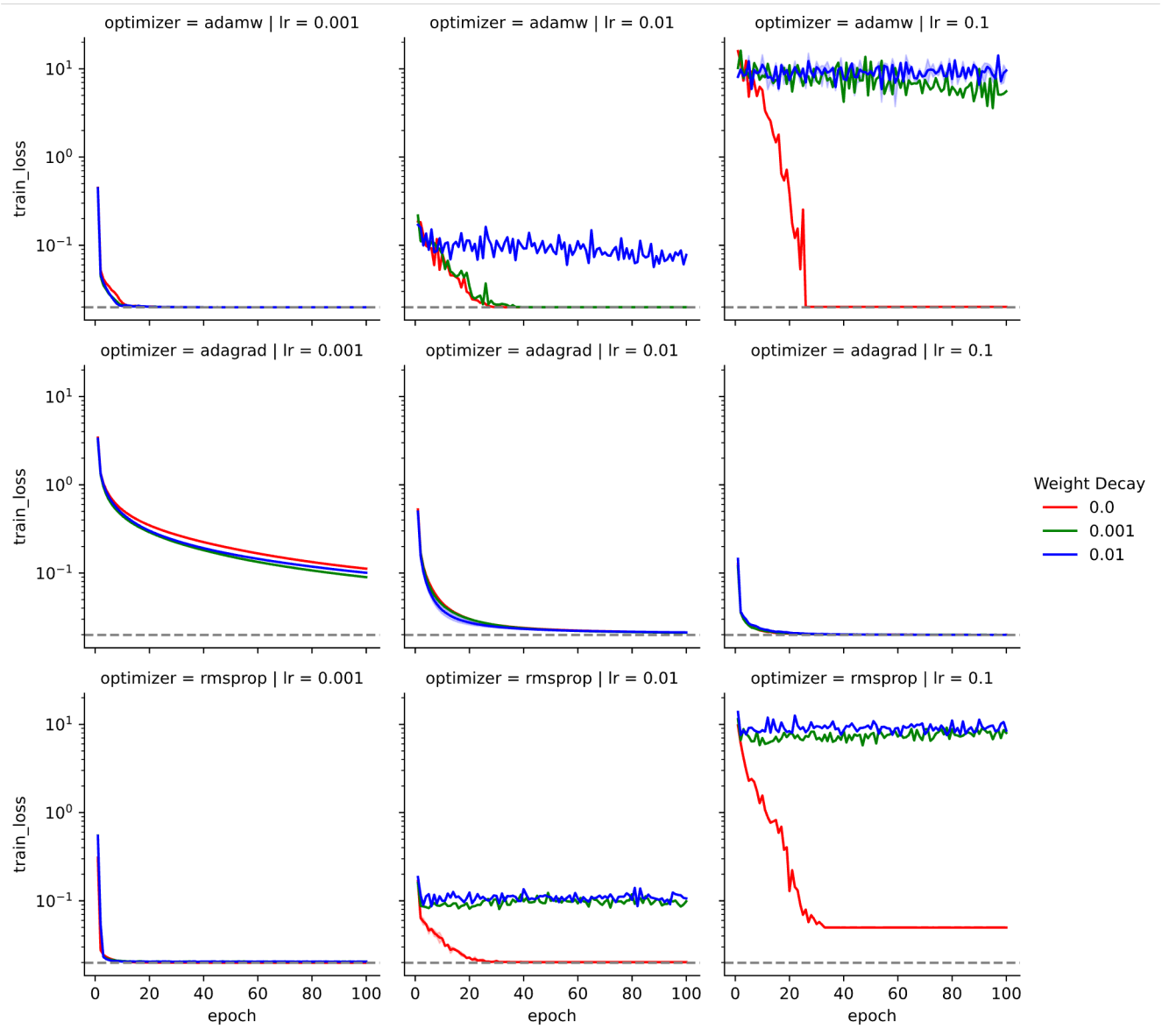
01 Optimizer Comparision

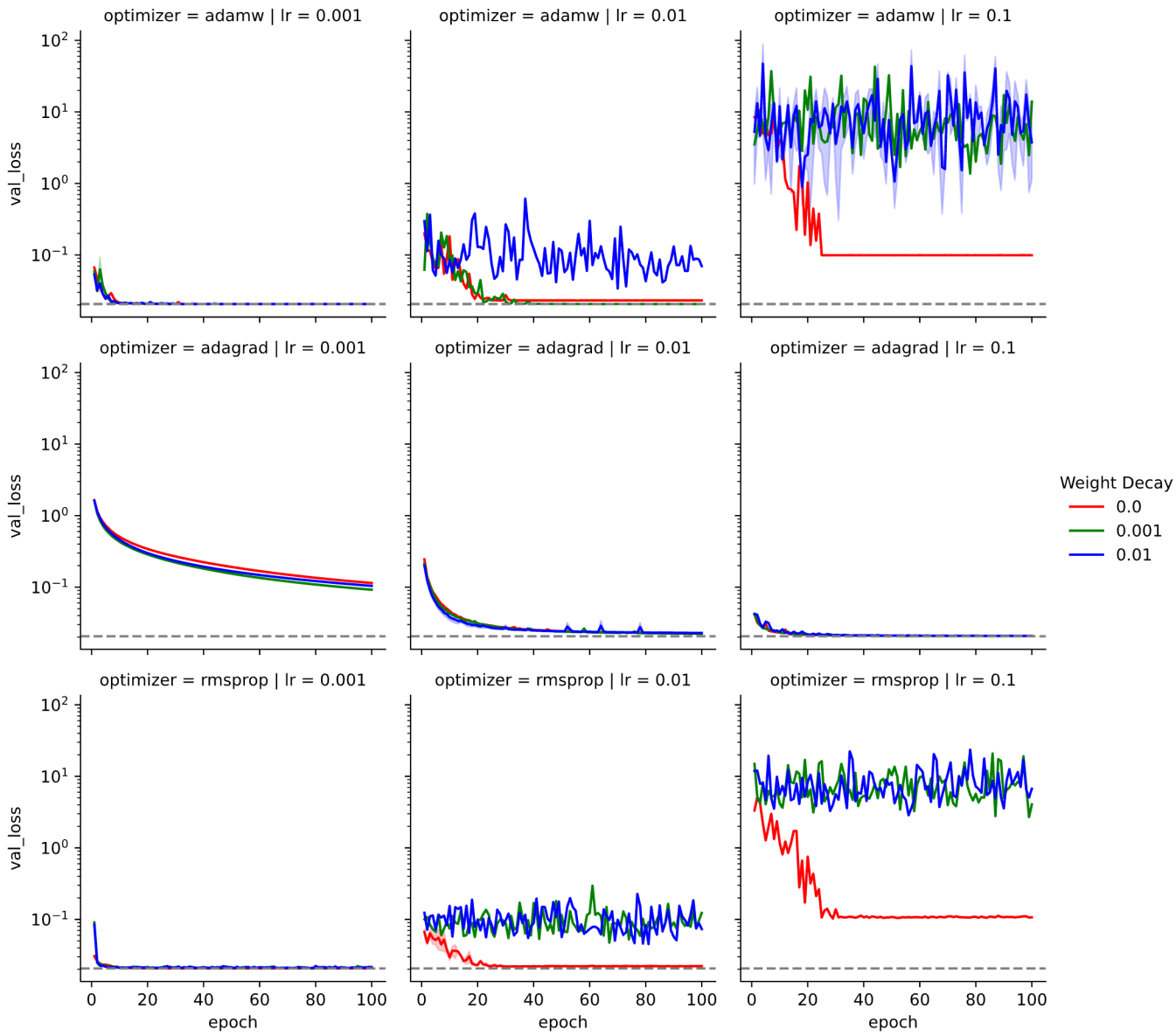
- Experiments were run for the following config with all combinations of optimizer parameters:
    - optimizer: RMSProp, AdaGrad, AdamW
    - learning rate: 0.1, 0.01, 0.001
    - weight decay: 0.01, 0.001, 0
- The dataset was randomly sampled 20% of the whole HGF (HourGlassForce) data. Training and validation was a random 80-20 split.
- Initialization was Kaiming Uniform and there was only one hidden layer of size 74 with a ReLU activation. Loss function was SmoothL1. Training was done in double float precision. Batch size was 1024, training was done for 100 epochs.

Training loss for various configurations. The lowest training loss was 0.01986



Validation loss for various configurations. The lowest validation loss was 0.02057

adagrad	0.020704
adamw	0.020574
rmsprop	0.020582



Following observations can be made from the training and validation plots:

- Training and Validation loss are closely following each other
- Best loss from all optimizers is similar (i.e. if we pick correct learning rate and weight decay the optimizer perform the same)
- Weight decay of 0.0 gives good result for any learning rate
- AdaGrad (middle row) seems more robust to learning rate and shows almost no variation due to weight decay
- AdamW (top row) and RMSProp (bottom row) perform well at lower learning rate (0.001) while AdaGrad converges faster for higher learning rate (0.01, 0.1).

The same experiments were done in single float and double float precision and the results were similar.

Precision	Min Train Loss	Min Validation Loss
Single	0.019861	0.020575
Double	0.019861	0.020574

02 A trick for training many models at once

import torch.multiprocessing as mp

Use same CPU memory for dataset (~4GB) then run multiple training processes that each train for different config. Each process requires only ~300 MB in CPU and ~150 MB in GPU.

```
elif shared_memory:
    print(' '.join([str(arg) for arg in params]))
    if dry_run:
        return

    p = mp.Process(target=train, args=(trial_config, dataset))
    p.start()
```

03 Initializer Comparision

Experiments were run for the following combinations of initialization:

- initialization:
    - Uniform (u),
    - Diagonally dominant Uniform (udd),
    - Kaiming Uniform (ku),
    - Xavier Uniform (xu)

for u and udd take initialization range [-1,1] and [-10, 10]

The dataset was randomly sampled 20% of the whole HGF (HourGlassForce) data. Training and validation was a random 80-20 split.

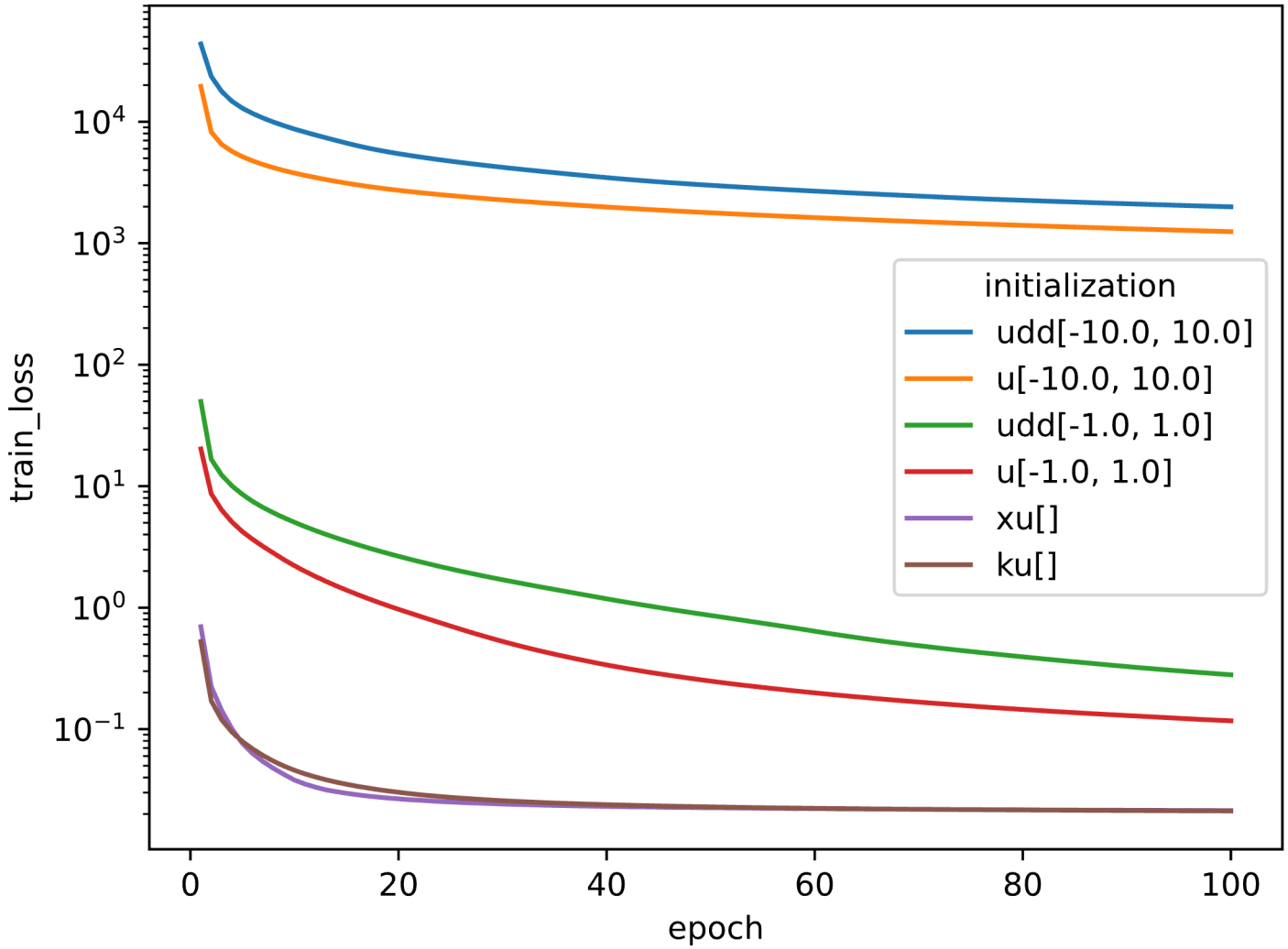
There was only one hidden layer of size 74 with a ReLU activation. Loss function was SmoothL1. Training was done in double float precision. Batch size was 1024, training was done for 100 epochs.

Based on results from optimization comparision AdaGrad with lr=0.01 and weight_decay=0 are used for the following comparisons.

Should have taken lr = 0.1

Kaiming (i.e. He) and Xavier (i.e. Golrot) initialization perform similarly while uniform initization in higher range [-1, 1] to [-10, 10] cause very slower convergence.

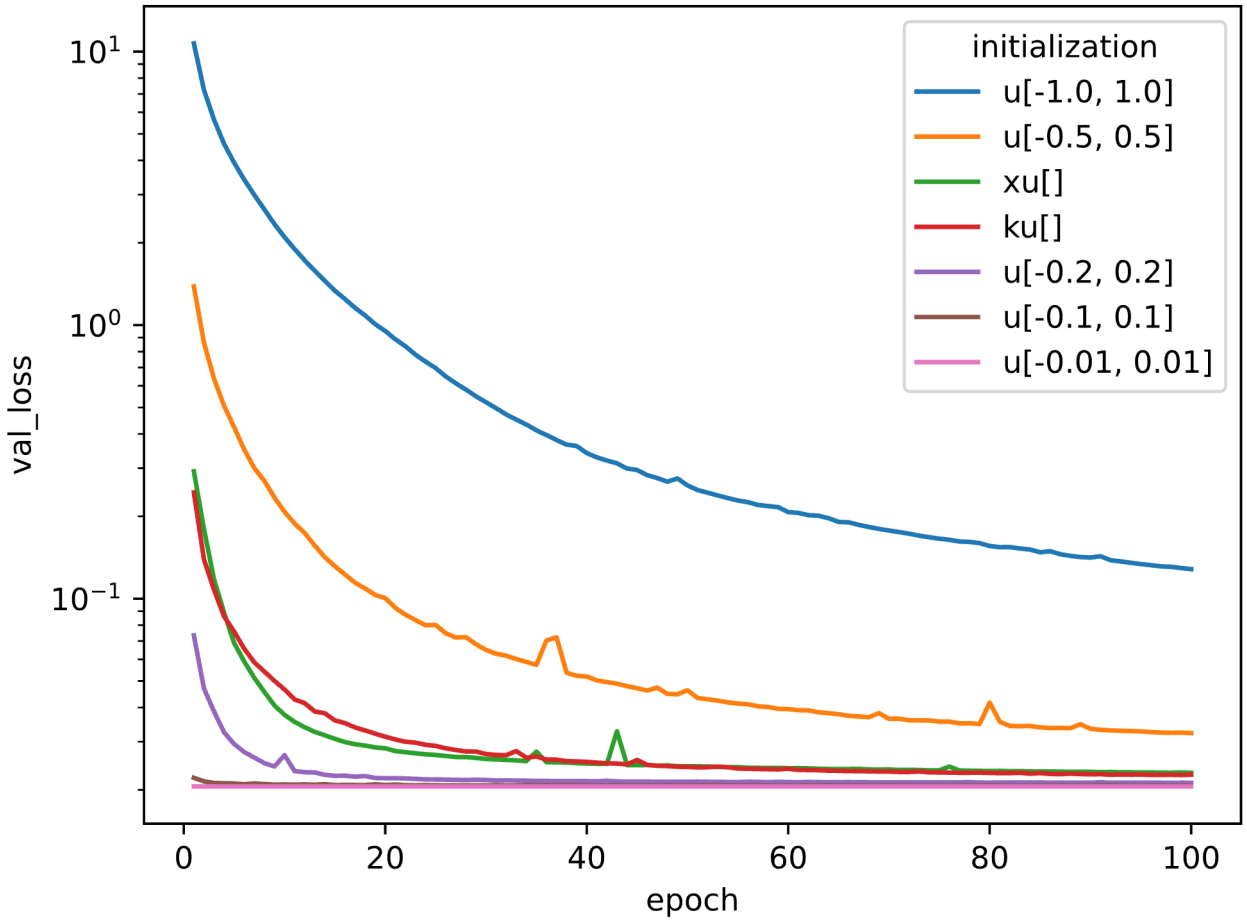
The validation loss curve is also similar to training loss curve.



Kaiming and Xavier are also uniform initialization that depend on the shape of the weight matrix (i.e. incoming and outgoing feature vector length). The equivalent uniform initialization for the weights in two layers are:

Initialization	W1	W2
ku	U[-0.285,0.285]	U[-0.288,0.288]
xu	U[-0.286,0.286]	U[-0.353,0.353]

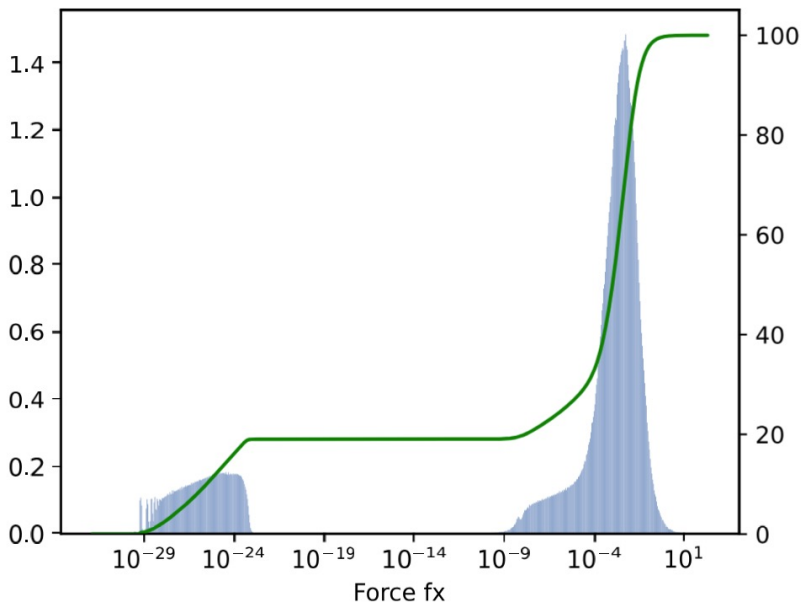
Further exploring uniform initialization



Why is lower better?

As the range of uniform initialization decreases the validation loss also improves.

This might be (I am not sure) because most of the output is in the $\leq 10^{-1}$ region. The average value of fx is -0.0015 ($\sigma = \pm 0.8$). So, small initialization give better results by default.

Initialization vs Validation Loss

initialization	
u[-0.01, 0.01]	0.020574
u[-0.1, 0.1]	0.020686
u[-0.2, 0.2]	0.021214
ku[]	0.022663
xu[]	0.023084
u[-0.5, 0.5]	0.032251
u[-1.0, 1.0]	0.128225
udd[-1.0, 1.0]	0.306875
u[-10.0, 10.0]	1246.092895
udd[-10.0, 10.0]	1981.471181